

표지

AI 기반 Kubernetes 장애 분석 및 복구 자동화 서비스

장애가 났을 때 흩어진 로그, 지표, 배포 변경 내역을 모아
원인을 설명하고 안전한 복구 방법을 제안하는 운영 보조 도구

303호 4팀

임가인, 이민정, 전우현, 정찬빈, 최우녕

멘토: 김시훈 멘토님

2026-06-29

이미지 방향: • Kubernetes 클러스터를 상징하는 간단한 배경 이미지 또는 직접 만든 구조 아이콘.
• 실제 대시보드가 없으므로 제품 화면처럼 보이는 가짜 UI 이미지는 사용하지 않는다.

한 장 요약

비전문가용 핵심 메시지

우리가 만들고 싶은 것은 “장애 대응 도우미”다.

상황 | 기존 방식 | 우리가 만들 방향

장애 발생 | 사람이 여러 도구를 직접 확인 | 시스템이 관련 증거를 자동으로 모음

원인 파악 | 경험 많은 사람이 로그와 지표를 해석 | AI가 근거와 함께 원인 후보를 설명

복구 방법 | 급하게 수동 명령 실행 | 사람이 승인한 안전한 명령 또는 PR 제안

기록 | 누가 왜 조치했는지 흩어짐 | 사건 흐름을 감사 로그로 남김

전문가 확인 포인트

- 핵심 흐름은 evidence → RCA → proposed action → approval → command/Safe PR → audit이다.
- 목표는 full auto remediation이 아니라 human-in-the-loop 운영 보조다.

현재 상태

- 이 전체 흐름은 아직 완전히 연결되지 않았다.
- 현재는 각 흐름을 담당할 서비스, 이벤트, WBS, 이슈를 쪼개고 일부 기반 구현을 검증 중이다.

필요 이미지

- “장애 발생 → 증거 수집 → AI 분석 → 승인 → 복구 제안 → 기록” 6단계 그림.
- 비전문가가 이해하기 쉽도록 도구명보다 사람의 행동 흐름 중심으로 그린다.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

왜 필요한가

비전문가용 핵심 메시지

Kubernetes 장애는 한 화면에서 끝나지 않는다.

장애가 나면 보통 이런 것들을 따로 봐야 한다.

- 어떤 코드나 설정이 바뀌었는지
- 어떤 Pod가 죽었는지
- 로그에 어떤 에러가 있는지
- CPU, 메모리, 재시작 횟수가 어떻게 변했는지
- 누가 어떤 명령을 실행했는지

문제는 이 정보들이 서로 다른 곳에 흩어져 있다는 점이다.

전문가 확인 포인트

- source of truth가 Git, Kubernetes API, Prometheus, Loki, event store, command queue로 분산된다.
- 프로젝트는 이 분산 데이터를 correlation_id 기준 사건 흐름으로 묶는 것을 목표로 한다.

현재 상태

- 실제 Prometheus/Loki 전체 연동은 아직 완료되지 않았다.
- 현재는 어떤 데이터를 Evidence로 줄일지 문서화하고, Prometheus demo/task를 준비 중이다.

필요 이미지

- 왼쪽에는 GitHub, Kubernetes, Prometheus, Loki, 사람이 흩어져 있는 그림.
- 오른쪽에는 우리 시스템이 가운데에서 증거를 모으는 그림.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

우리가 해결하려는 사용자 문제

비전문가용 핵심 메시지

사용자는 “왜 장애가 났고, 지금 무엇을 해야 하는지”를 알고 싶다.

사용자 | 어려움 | 우리가 주려는 도움

개발자 | 내 변경이 장애와 관련 있는지 모름 | 관련 commit, 로그, 지표를 함께 보여줌

운영자 | 여러 도구를 보며 원인을 추정해야 함 | 원인 후보와 근거를 정리

팀 리더 | 누가 어떤 판단으로 조치했는지 보기 어려움 | 사건 흐름과 조치 이력을 남김

전문가 확인 포인트

- persona는 developer, SRE/operator, team lead로 나눌 수 있다.
- dashboard는 아직 없지만 read model과 audit timeline 설계가 이 요구를 받는다.

현재 상태

- UI dashboard는 아직 없다.
- 현재는 Gateway API, dashboard read model, audit event로 연결할 계획을 문서화한 상태다.

필요 이미지

- 세 사용자 카드: 개발자, 운영자, 팀 리더.
- 각 카드에 “궁금한 질문” 말풍선 추가.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

우리가 만들 최종 흐름

비전문가용 핵심 메시지

최종적으로는 아래 흐름을 목표로 한다.

장애 징후 발생

- 시스템이 로그/지표/이벤트/배포 변경 수집
- AI가 원인 후보와 근거 설명
- 사람이 복구 방법 승인
- 안전한 명령 실행 또는 GitHub PR 생성
- 결과와 판단 과정을 기록

전문가 확인 포인트

- event-driven workflow: cluster.evidence.received, evidence.built, rca.completed, safe_pr.requested, command.*.
- command execution은 sandbox namespace와 policy guard를 거친다.
- Safe PR은 repo-gateway-worker 같은 outbound gateway로 분리한다.

현재 상태

- 이 흐름은 목표 구조다.
- 현재는 전체 사이클이 연결되지 않았다.
- Docker/kind 환경에서 management/target 분리, event runtime, DLQ, target telemetry 일부를 준비 중이다.

필요 이미지

- 목표 흐름도.
- 각 단계 아래에 “현재: 준비 중 / 미완성 / 일부 구현” 배지를 표시하면 좋다.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

지금 실제로 된 것과 안 된 것

비전문가용 핵심 메시지

현재는 완성 화면을 만든 단계가 아니라, 시스템의 뼈대를 만드는 단계다.

구분 | 상태

Docker/kind 환경 분리 | 관리 클러스터와 타깃 클러스터를 나누는 방향으로 작업 중

이벤트/큐 시스템 | 설계와 일부 구현/테스트 진행

로그인/Gateway | email/password + Redis session 브랜치 진행

Target Agent/Telemetry | Prometheus/Node Collector 학습 task와 일부 브랜치 진행

전체 장애 대응 사이클 | 아직 미연결

대시보드 UI | 아직 없음

실제 AI RCA 고도화 | 아직 계획/기초 설계 단계

전문가 확인 포인트

- 현재 보고서에서 `완료`라는 표현은 조심해서 써야 한다.
- local/branch 테스트 증거와 merge된 main/dev 상태를 구분해야 한다.

현재 상태

- 전체 end-to-end demo는 아직 성공 상태로 단정하면 안 된다.
- 현재는 2026-07-01 1차 보고를 위해 event runtime, DLQ, Node Collector를 우선 연결해야 한다.

필요 이미지

- 진행 상태 보드.
- 완료/진행/미완성/계획을 색으로 구분한 표.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

현재 개발 환경

비전문가용 핵심 메시지

우리는 실제 운영 환경을 바로 쓰지 않고, 로컬에서 작은 Kubernetes 환경을 나눠 테스트한다.

관리 환경

- API Gateway
- 이벤트 큐
- 작업자 서비스
- 데이터베이스

타깃 환경

- 장애가 난다고 가정하는 Kubernetes 클러스터
- Target Agent
- Node Collector

전문가 확인 포인트

- management kind cluster와 target kind cluster 분리.
- management에는 NATS, PostgreSQL, Redis, worker들이 있고 target에는 target-cluster-agent, node-collector, fake telemetry가 배치된다.
- target agent는 management NATS에 직접 붙지 않고 Gateway HTTP로 outbound 통신해야 한다.

현재 상태

- Docker/kind 기반 분리 환경을 기준으로 설계/검증 중이다.
- 실제 cloud/EKS 배포는 2주차 이후 후보로 둔다.

필요 이미지

- 두 개의 박스: Management Cluster / Target Cluster.
- 두 박스 사이에 한 방향 화살표: Target Agent → Gateway.
- Docker/kind 로고나 터미널 `kind get clusters` 캡처가 있으면 좋다.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

시스템을 큰 덩어리로 보면

비전문가용 핵심 메시지

시스템은 크게 5개 덩어리로 나눌 수 있다.

덩어리 | 쉬운 설명

Gateway/Auth | 외부 요청이 들어오는 정문

Event/Queue | 서비스끼리 일을 넘겨주는 작업 줄

GitOps/Command | 배포 변경을 보고 안전한 명령으로 바꾸는 곳

Target/Telemetry | Kubernetes 상태와 지표를 가져오는 곳

RCA/Safe PR/Audit | 원인을 정리하고 PR/기록으로 남기는 곳

전문가 확인 포인트

- 각 덩어리는 독립 서비스 또는 worker group이다.
- 서비스 간 직접 함수 호출 대신 NATS JetStream event, Gateway HTTP, DB queue를 사용한다.

현재 상태

- 구조와 역할은 문서화되어 있다.
- 각 서비스가 완전히 통합되어 실행되는 상태는 아직 아니다.

필요 이미지

- 5개 큰 블록을 원형 또는 좌우 흐름으로 배치.
- 각 블록에 담당자 이름을 작게 표시.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

Event/Queue를 쉽게 설명하면

비전문가용 핵심 메시지

이벤트 큐는 “작업 전달함”이다.

예를 들어:

1. Gateway가 “명령 요청이 들어왔다”고 전달함에 넣는다.
2. Command Worker가 그 일을 꺼내 정책을 확인한다.
3. 문제가 없으면 “에이전트가 가져갈 명령이 준비됐다”고 다시 전달함에 넣는다.
4. Target Agent가 가져가 실행 결과를 다시 알려준다.

전문가 확인 포인트

- NATS JetStream durable pull consumer.
- at-least-once delivery이므로 handler와 DB write는 idempotent해야 한다.
- `event\_processing(event\_id, consumer)` ledger, bounded retry, DLQ/replay가 필요하다.

현재 상태

- Event runtime, envelope, subject/body 계약, DLQ/replay 구조를 설계/구현 중이다.
- local test는 있으나 전체 환경 smoke는 다시 확인해야 한다.

필요 이미지

- 우체통/작업함 비유 그림.
- “요청 이벤트 → 작업자 → 결과 이벤트” 단순 흐름도.
- 전문가용 appendix에는 NATS/DLQ 구조를 작게 추가.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

왜 AI가 바로 고치면 안 되는가

비전문가용 핵심 메시지

AI가 바로 운영 서버를 고치면 위험하다.

그래서 우리는 AI를 “자동 실행자”가 아니라 “근거를 정리하고 제안하는 보조자”로 둔다.

위험 | 우리 방식

AI가 잘못 판단할 수 있음 | 사람이 승인해야 실행
운영 서버를 망가뜨릴 수 있음 | sandbox 영역부터 제한
왜 고쳤는지 모를 수 있음 | Evidence와 audit 기록 남김
코드 변경이 Git 기록과 어긋날 수 있음 | Safe PR로 제안

전문가 확인 포인트

- human-in-the-loop, approval guard, sandbox namespace, Safe PR.
- command policy와 credential boundary가 핵심이다.

현재 상태

- AI RCA 고도화는 아직 완성되지 않았다.
- 현재는 rule 기반 RCA baseline과 Evidence 구조부터 준비하는 단계다.

필요 이미지

- AI가 직접 서버를 고치는 그림에 금지 표시.
- AI가 보고서를 만들고 사람이 승인하는 그림.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

Evidence를 쉽게 설명하면

비전문가용 핵심 메시지

Evidence는 “AI가 원인을 판단할 때 보는 증거 묶음”이다.

raw data는 너무 많다.

예:

- 로그 1만 줄
- 지표 수천 개
- Kubernetes 이벤트 수십 개

Evidence는 이것을 줄여서 이렇게 만든다.

- 어떤 Pod가 죽었는지
- 언제부터 재시작이 늘었는지

전문가 확인 포인트

- Evidence는 event payload 계약으로 확정되기 전 내부 중간 모델로 다룬다.
- Kubernetes Pod/Event, Prometheus metric, Loki log, OTel trace reference를 source별로 normalize해야 한다.

현재 상태

- Evidence 모델과 학습 문서를 작성했다.
- 실제 Prometheus/Loki/OTel 연동은 아직 단계적으로 붙일 예정이다.

필요 이미지

- 왼쪽: 큰 raw data 더미.
- 오른쪽: 작은 Evidence 카드 4개.
- 각 카드에 Pod, Metric, Log, Git Change 아이콘.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

Prometheus/Loki/OTel을 쉽게 설명하면

비전문가용 핵심 메시지

장애를 이해하려면 여러 종류의 데이터를 봐야 한다.

도구 | 쉬운 설명 | 보는 것

Prometheus | 숫자 지표를 모으는 도구 | CPU, 메모리, 재시작 횟수

Loki | 로그를 모으는 도구 | 에러 메시지, 실행 로그

OpenTelemetry | 요청 흐름을 추적하는 표준 | 어떤 요청이 어디서 느려졌는지

전문가 확인 포인트

- Prometheus는 pull/scrape 모델.
- Loki와 OTel은 ingest/push 모델도 강하다.
- Node Collector `/metrics`는 Prometheus scrape target이 될 수 있다.

현재 상태

- 아직 전체 관측성 스택이 완성된 것은 아니다.
- 우선 Node Collector `/metrics`와 Prometheus scrape/query 폐쇄 루프를 만드는 것이 첫 목표다.

필요 이미지

- Prometheus는 “데이터를 가지러 가는 화살표”.
- Loki/OTel은 “데이터를 보내는 화살표”.
- pull/push 방향 차이를 그림으로 표현.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

Gateway/Auth를 쉽게 설명하면

비전문가용 핵심 메시지

Gateway는 우리 서비스의 정문이다.

정문에서 확인할 것:

- 누구인가?
- 어떤 프로젝트에 접근할 수 있는가?
- 어떤 외부 도구를 연결했는가?
- 어떤 명령을 실행해도 되는가?

전문가 확인 포인트

- OAuth-only가 아니라 internal identity + server-side Redis session.
- organization/project RBAC와 integration target/credential binding 분리.
- Token Broker는 worker가 secret을 직접 읽지 않게 하는 경계다.

현재 상태

- email/password + Redis session 브랜치가 있다.
- 하지만 브랜치가 dev보다 behind이고 `app.py` 기준 구조와 정렬이 필요하다.
- org/project RBAC와 Token Broker는 아직 단계적으로 구현해야 한다.

필요 이미지

- 건물 정문/출입증 비유 그림.
- “User / Project / Integration / Credential” 4단계 계층도.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

GitOps/Command를 쉽게 설명하면

비전문가용 핵심 메시지

GitOps는 “운영 상태를 Git에 기록된 원하는 상태와 맞추는 방식”이다.

우리 프로젝트에서는:

1. Git 저장소를 주기적으로 확인한다.
2. 변경된 설정을 읽는다.
3. 실제 클러스터 상태와 비교한다.
4. 차이가 있으면 안전한 명령 또는 PR 후보를 만든다.

전문가 확인 포인트

- 기본 방향은 webhook이 아니라 polling-first.
- git pull worker, manifest render worker, diff worker, diff analyze worker, repo gateway worker로 분리한다.
- command.requested와 safe_pr.requested 분기 기준이 필요하다.

현재 상태

- GitOps polling API 설계 문서 PR은 merge되었다.
- 실제 GitOps/Command 구현 브랜치는 아직 초기 상태다.

필요 이미지

- Git repository와 Kubernetes cluster를 비교하는 그림.
- “Git 상태”와 “실제 상태” 사이의 diff 표시.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

RCA/Safe PR을 쉽게 설명하면

비전문가용 핵심 메시지

RCA는 “왜 문제가 생겼는지 정리한 원인 분석”이다.

Safe PR은 “바로 고치지 않고, 안전하게 검토할 수 있는 수정 제안”이다.

Evidence

- RCA 요약
- 수정 제안
- GitHub PR 초안
- 사람이 검토 후 merge

전문가 확인 포인트

- RCA Worker는 GitHub PR을 직접 만들지 않고 `safe\_pr.requested`를 발행한다.
- repo-gateway-worker가 외부 provider 호출을 담당한다.
- Safe PR body에는 원인, 근거, 변경 내용, 검증 방법, rollback이 들어가야 한다.

현재 상태

- RCA/Safe PR 구조는 문서와 일부 source 기준으로 정리되어 있다.
- 담당 원격 브랜치에는 아직 실제 구현 diff 반영이 부족하다.

필요 이미지

- 사고 보고서에서 PR로 이어지는 흐름도.
- GitHub PR 화면 예시는 실제 PR이 없으면 mock이 아니라 “예상 PR 구성” 박스로 표현.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

대시보드는 아직 없다

비전문가용 핵심 메시지

현재 대시보드 화면은 아직 없다.

대신 먼저 해야 하는 일:

1. 어떤 사건 정보를 모을지 정한다.
2. 그 정보를 저장할 read model을 만든다.
3. Gateway API로 조회할 수 있게 한다.
4. 이후 UI가 이 API를 보여준다.

전문가 확인 포인트

- dashboard-projection-service는 모든 주요 event를 read model로 투영한다.
- audit-timeline-service는 command/evidence/RCA/PR 흐름을 correlation_id로 보존한다.
- UI는 DB/JetStream에 직접 붙지 않는다.

현재 상태

- dashboard UI는 미구현이다.
- dashboard read model과 query/stream 계약을 설계해야 한다.

필요 이미지

- 실제 없는 UI 스크린샷 대신 “예상 화면 와이어프레임”을 사용.
- 예: 사건 카드, 원인 요약, 증거 목록, 제안 PR, command 상태.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

이번 주 실제 성과

비전문가용 핵심 메시지

이번 주에는 완성 화면보다 “팀이 같은 방향으로 구현할 수 있는 기반”을 만들었다.

영역 | 성과

프로젝트 정의 | 무엇을 만들고, 무엇을 하지 않을지 정리

멘토링 반영 | MVP 범위, 저장소, 인증, AI 역할 재정리

WBS | 56개 작업과 수요일 데모 마일스톤 정리

팀 역할 | 5개 담당 영역과 각자 이슈 분리

Event System | 이벤트/큐/DLQ/replay 구조 정리

Target/Telemetry | Evidence 학습 문서와 Prometheus task 정리

전문가 확인 포인트

- PR #538~#541이 merge되어 문서 반영은 일부 main/dev에 들어갔다.
- 구현 이슈 #501~#509, #519~#537이 열려 있다.

현재 상태

- 구현 완료보다 설계/문서/WBS/이슈 분해 성과가 큰 주차다.

필요 이미지

- GitHub PR merged 목록 캡처.
- GitHub Project/WBS 또는 Issues 목록 캡처.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

지금 가장 큰 리스크

비전문가용 핵심 메시지

현재 가장 큰 위험은 “설계는 있는데 아직 한 번에 동작하는 흐름이 없다”는 점이다.

리스크 | 쉽게 말하면 | 대응

전체 사이클 미연결 | 끝까지 돌아가는 demo가 아직 없음 | 작은 fake 흐름부터 연결

대시보드 없음 | 보여줄 화면이 없음 | API/curl demo를 먼저 준비

외부 도구 연동 어려움 | GitHub/Prometheus가 바로 안 붙을 수 있음 | fake adapter와 dry-run 유지

브랜치 충돌 | 각자 만든 코드가 합쳐지며 깨질 수 있음 | dev 기준 재정렬

팀원 이해도 차이 | 기술 용어가 어려움 | 문서에 쉬운 설명과 첫 task 제공

전문가 확인 포인트

- Gateway branch behind, Target path mismatch, RCA remote diff 부족, smoke 미검증이 구체 리스크다.
- next action은 구현보다 integration hygiene도 중요하다.

현재 상태

- 이 리스크들은 아직 해소되지 않았다.
- 다음 주 초 작업의 우선순위가 된다.

필요 이미지

- 위험 신호 표.
- “현재 위치 → 7/1 데모 목표” 거리감을 보여주는 로드맵.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

다음 주 목표

비전문가용 핵심 메시지

다음 주 목표는 “작아도 실제로 돌아가는 한 사이클”이다.

우선 보여줄 것:

1. Docker/kind 환경 실행
2. 이벤트가 발행되고 worker가 처리하는 것
3. 실패 이벤트가 DLQ로 가는 것
4. DLQ를 다시 replay하는 것
5. Node Collector가 `/metrics`를 보여주는 것
6. 가능하면 command 또는 evidence가 Gateway를 통해 흐르는 것

전문가 확인 포인트

- `make up && make smoke && make status` 재검증.
- DLQ 생성/조회/replay 경로 고정.
- Gateway/Auth branch rebase, Target path correction, RCA remote diff 반영.

현재 상태

- 2026-07-01 1차 보고 전까지 실제 실행 로그가 필요하다.

필요 이미지

- 터미널 실행 캡처: `make up`, `make status`, worker logs.
- DLQ API curl 결과 캡처.
- Node Collector `/metrics` curl 결과 캡처.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

멘토링 피드백 반영

비전문가용 핵심 메시지

멘토님 피드백은 “크게 만들기보다, 돌아가는 한 사이클을 먼저 만들자”였다.

피드백 | 반영

MVP는 작게, 실제로 돌아가게 | 7/1 데모를 Event/DLQ/Node Collector 중심으로 조정

인증은 너무 커질 수 있음 | 내부 로그인 + session부터 시작

데이터는 목적이 먼저 | Evidence와 RCA 중심으로 정리

AI는 직접 고치는 존재가 아님 | 제안하고 사람이 승인하는 구조

secret은 절대 노출 금지 | event/log/response에는 ref만 전달

전문가 확인 포인트

- 멘토링 결정은 WIKI 회의록, Gateway/Auth guide, Target/Telemetry docs, WBS에 반영되었다.

현재 상태

- 반영은 주로 문서와 이슈 설계 단계다.
- 코드 반영은 각 담당 브랜치에서 이어져야 한다.

필요 이미지

- 멘토링 회의록 일부 캡처.
- “피드백 → 문서/이슈 반영” 연결 표.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

자체 평가

비전문가용 핵심 메시지

구분 | 평가

잘한 점 | 큰 프로젝트를 역할과 작은 작업으로 나누었다.

잘한 점 | 멘토링 피드백을 문서와 이슈에 반영했다.

아쉬운 점 | 아직 전체 사이클과 대시보드는 연결되지 않았다.

아쉬운 점 | 기술 설명이 어려워 비전문가에게는 더 쉬운 자료가 필요하다.

다음 보완 | 실행 가능한 demo와 쉬운 그림 중심 보고서로 바꾼다.

전문가 확인 포인트

- 설계 문서와 실제 branch 상태 사이의 gap을 줄여야 한다.
- 완료 정의는 code + test + docs + project status가 함께 맞아야 한다.

현재 상태

- 현재는 “기반 정리 주차”로 평가한다.
- 다음 보고서에서는 실행 증거를 더 많이 넣어야 한다.

필요 이미지

- 완료/진행/미완성 비율 막대.
- 다음 주 목표 체크리스트.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용

Appendix - 실제 근거 자료

비전문가용 핵심 메시지

이 보고서는 아래 자료를 근거로 작성했다.

종류 | 자료

멘토링 이메일 | `Gmail - Jungle Mentor Meet\_.pdf`

멘토링 회의록 | WIKI `2026-06-26-mentoring.md`

MVP 계획 | WIKI `mvp-plan.md`

WBS | WIKI `wbs.md`, source WBS.xlsx

아키텍처 | source `docs/architecture.md`

이벤트 시스템 | source `docs/events.md`

팀 작업 분배 | source `docs/team/work-allocation.md`

GitHub 상태 | PR #538~#541, Issues #501~#509, #519~#537

전문가 확인 포인트

- 자료 출처와 실제 파일 경로는 MD 원고에 남긴다.

현재 상태

- 이 appendix는 제출용 PPT에서는 줄일 수 있지만, 멘토링/KDT 히스토리에는 남기는 것이 좋다.

필요 이미지

- GitHub PR/Issue 화면 캡처.
- WIKI 회의록 캡처.

이미지 자리: 실제 대시보드가 없으므로 개념도/흐름도/터미널 로그/GitHub 캡처를 사용